On Genomic CGR-images classification by Spiking Neural P Systems

Alessia Ruggiero

1 Methodology

1.1 Spiking Neural (SN) P Systems

A SN P system incorporates the structure of a neural network with the paradigm of membrane computing to replicate the way in which neurons communicate through spikes. It consists of a set of neurons, which represent cells made of a single membrane, placed in a directed graph; neurons can fire signals (*spikes*) along the edges of the graph, also referred to as 'synapses'. Neurons that can only fire spikes to the environment are called *output neuron*. Each neuron evolves following *spiking rules*, which are represented in the form

$$E/a^c \rightarrow a; d \tag{1}$$

where E is a regular expression over the spikes of the cell (denoted by a) and c,d are natural numbers (c≥1, d≥0). This means that a neuron can produce a spike if it contains k spikes, with $E = a^k$,$k \geq c$, by consuming c spikes, with a delay of d steps. If $d \geq 1$, the node is 'closed' for the next d steps, where it cannot receive any new spike. Additionally, the spike produced by a spiking rule is sent to all neurons connected through a synapse to the firing neuron. There exist also *forgetting rules*, represented by

$$a^s \rightarrow \lambda \tag{2}$$

meaning that s spikes are forgotten.
At each time, all neurons that can use a rule should do it, with at most one rule per neuron, which, depending on the type of system, can be chosen randomly or non-deterministically if there are more than one possibility.

Formally, a SN P system is a construct of the type

$$\Pi = (O, \sigma_1, ..., \sigma_m, syn, out) \tag{3}$$

where $O = \{a\}$ (a is a spike), $\sigma_1, ..., \sigma_m$ are the neurons, $syn \subseteq \{1, 2, ..., m\} \times \{1, 2, ..., m\}$ are the synapses and $out \in \{1, 2, ..., m\}$ is the set of output neuron. Additionally, each neuron is represented by

$$\sigma_i = (n_i, R_i)$$

where $n_i \geq 0$ is the initial number of spikes of the neuron, and R_i is the set of rules (both spiking and forgetting).

The initial configuration of the system is represented by the number of spikes present in each neuron. In general, a *configuration* describes the number of spikes in each neuron and whether a neuron is open o closed at a certain time. A *computation* is therefore defined as any sequence of transitions of configurations starting from the initial configuration. The output of a computation is represented by the number of spikes directed towards the environment [1].

1.1.1 Example

The SN P system

$$\Pi_1 = (\{a\}, \sigma_i, \sigma_2, \sigma_3, syn, 3)$$

where

$$\sigma_1 = (2, \{a^2/a \to a; 0, a \to \lambda\})$$

$$\sigma_2 = (1, \{a \to a; 0, a \to a; 1\})$$

$$\sigma_3 = (3, \{a^3 \to a; 0, a \to a; 1, a^2 \to \lambda\})$$

$$syn = \{(1, 2), (2, 1), (1, 3), (2, 3)\},$$

is a system able to generate all natural numbers greater than 1. Figure 1 shows a graphical representation of the initial configuration.

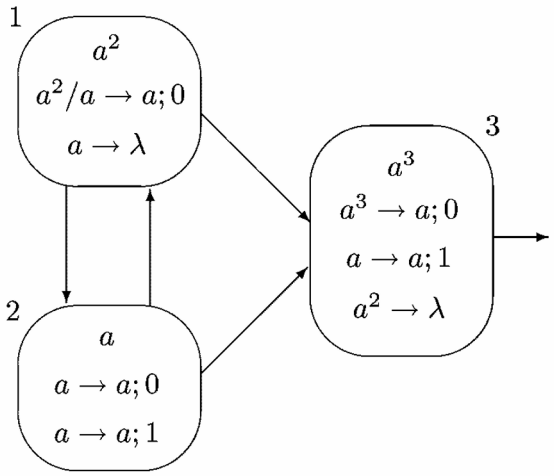


Figure 1: SN P system of 3 neurons able to generate all natural numbers greater than 1.

1.2 Chaos Game Representation (CGR)

The CGR is the result of the homonym algorithm, which generates images from a given input sequence. The algorithm proceeds in this way:

1. locate n vertices; these vertices may represent the alphabet from which the sequence was generated, or groups of such characters.

2. start from a point inside the region delimited by the vertices (can easily be the middle); scan the sequence, and for each character place a point half-way between the previous point and the vertix corresponding to the current character.

If this procedure is repeated for long random sequences (thousands of characters), fractals appear, but only if the number of vertices is 3, 5, 6 or 7. With 4, for example, the chaos game algorithm on such random strings produces a square uniformly and randomly filled with dots [2]. Figure 2 shows the chaos game representation of two random strings, from alphabets with cardinality of respectively 3 and 4.

Clearly, such input sequences can be obtained from any type of alphabet. It follows that DNA sequences, which can be formally defined as sequences of nucleotides $\{A, C, G, T\}$, can be represented using the chaos game representation. Interestingly, when applying chaos game to real DNA sequences using 4 vertices (one for each nucleotide), patterns appear in the CGR; these patterns reflect the patterns of repeated substrings inside the sequence. Figure 3 shows an example.

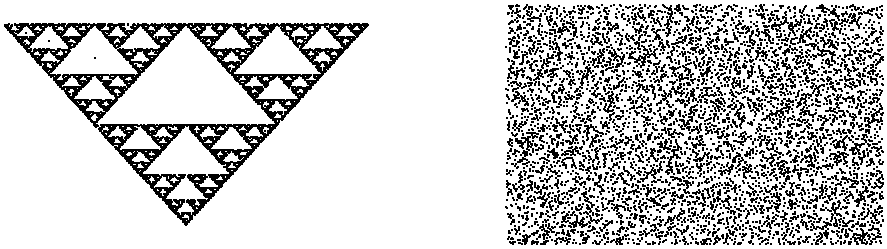


Figure 2: Chaos game representation of random strings of size 10,000, with characters drawn respectively from $\{1, 2, 3\}$ and $\{1, 2, 3, 4\}$, obtained with the R package kaos

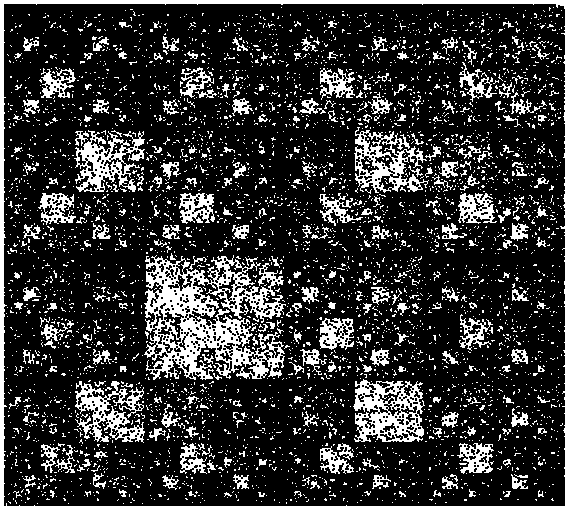


Figure 3: CGR of *Homo sapiens* EGFR (Epithelial Growth Factor Receptor) gene, generated with the R package kaos.

A less noisy representation using the chaos game paradigm is the frequency chaos game representation; FCGR is based on counting the points of the CGR in a pre-defined grid $n \times n$, where n is the resolution, thus representing the frequency of k-mers inside the sequence. A matrix of values is therefore obtained, which can be visualized, for example, in grayscale. Figure 4 shows an example of FCGR.

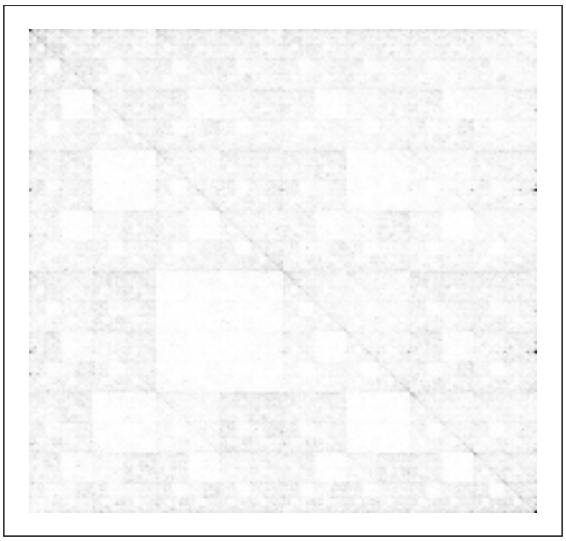


Figure 4: FCGR of *Homo sapiens* EGFR gene, with a resolution of 200x200, using R package kaos.

FCGR, like CGR, can be used to visualize homology between genomes. A major application of this method in bioinformatics is, indeed, the alignment-free comparison; the vectorization of both CGR and FCGR correspond to the k-mer frequencies in the sequence, information that have been widely exploited in alignment-free methods as it represents a major global property of DNA sequences. Several approaches exist in this context; for instance, using a distance measure (eg. Euclidean, Manhattan, Pearson..) to compare FCGR matrices, or applying machine learning methods for image classification of both FCGR and CGR [3].

1.3 An SN P System for image recognition

As a result of the collaboration between the University of Milano-Bicocca and the University of Verona, a *Python* implementation of a SN P system that recreates the main mechanisms of a Convolutional Neural Network (CNN) to perform an image classification was produced (https://github.com/SandroErba/From-CNN-to-SNPS)[4]. The model represents an extended SN P system, where more that one spike is allowed to fire at a time step, with a

white mode feature, meaning that when a neuron fires all its spikes are consumed. Additionally, inhibitory synapses are present, which are able to reduce the number of spikes a target neuron presents. In particular, spikes traversin an inhibitory synapse are acquired from the target neuron in the from of anti-spikes.
Formally, these SN P systems are described by

$$\Pi = (O, \sigma_1, \sigma_2, ..., \sigma_m, syn, in, out) \tag{4}$$

where $O = \{a, \bar{a}\}$ are the spikes (a denotes a spike and $\bar{a}$ and anti-spike), $\sigma_2, ..., \sigma_m$ are neurons of the form $\sigma_i = (n_i, R_i)$. The set of rules R_i can contain:

- Extended white-hole spiking rule: $E/b^{all} \rightarrow b^p; d$, where $b \in \{a, \bar{a}\}$; this means that if the neuron contains k spikes (or anti-spikes) b such that $b^k \in L(E)$, then all b spikes are consumed, and p are emitter through the synapses after d time units.
- White-hole forgetting rule: $b^{all} \rightarrow \lambda$, with $b \in \{a, \bar{a}\}$, which is applied when $b^{all} \notin L(E)$ for any spiking rule. In this case, all spikes are removed from the neuron.

Then, $syn = syn_e \cup syn_i \subseteq \{1, 2, ..., m\} \times \{1, 2, ..., m\}$ is the set of synapses, where syn_e is the set of excitatory synapses and syn_i is the set of inhibitory synapses; spikes transmitted through inhibitory synapses are interpreted as anti-spikes. Finally, in and out denote the sets of input and output neurons.

Considering input images of size 28×28, the structure of the implemented extended SN P system mimicking a CNN is the following:

- **Layer 1**: *Input encoding and feature extraction*. The input image (28×28) is mapped into the neurons, with each neuron corresponding to a distinct pixel. The size is therefore of $28 \cdot 28 = 784$ neurons. Firing rules convert pixel intensities (ranging from 0 to 255) into a proportional number of spikes, depending on the the parameter q-range.
- **Layer 2**: *Convolution and forgetting mechanism* . Layer 2 starts with eight feature maps from Layer 1 (convolutional kernels), which aim at representing the orientation of the input image in the form of a 3×3 matrix. In particular, the values of this matrix determine the type of synapse between two neurons: excitatory (1), inhibitory (-1) or absent (0). Layer 2 is therefore composed of $(28-2) \cdot (28-2) \cdot 8 = 5408$ neurons. Each neuron is equipped with firing rules that simply emit all the spikes currently stored in the neurons. Each neuron in Layer 2 has exactly one outgoing synapse, which is always

excitatory; neurons are organized in groups of four, such that each group is linked to a single neuron in Layer 3.

- **Layer 3**: *Average pooling, neuron importance, and training.* Layer 3 consists of 13·13·8 = 1352 neurons, as each neuron is formed by aggregating 4 neurons from Layer 2. Each neuron emits approximately one fourth of its internal charge, effectively reducing the spikes from the previous layer. This layer is directly connected to the final classification layer, and it represents the only component where a training phase is performed. For each input sample, the internal charges of the 1352 neurons in Layer 3 are collected; a matrix of size 1352 × N is therefore obtained, where N is the number of images in the training set. The matrix is used as input for a Support Vector Machine and a Logistic Regression model. Both models learn to map the features (1352) into the corresponding class labels. As a result of the training process, each model produces a real weight matrix, corresponding to the contribution of each neuron to each output class. As two models are employed, neurons in Layer 3 are doubled. Withing the weight matrix, an importance score, which can range from 1 to a used defined threshold, is assigned to each neuron of Layer 3 (separately for the two models), which allows to distinguish neurons that actually contibute to the classification from those that introduce noise.

- **Layer 4**: *Classification.* The final classification is performed using the importance-scaled outputs from the multiple classifiers. Output neurons (one per class) accumulate incoming spikes and fire according to their firing rules; the neuron emitting the highest number of spikes determines the predicted class.

The implementation can effectively classify images from the MNIST dataset [5]; the accuracy reaches 0.95 when the SVM weight matrix is employed.

A configuration file allows to change the parameters of the analysis. Among them there are

1. **Image shape:** must be set based on the size of the input images
2. ***q*-range:** also referred to as quantization range, it controls the number of spikes used to represent the input. Low values can degrade the performance of all models due to a loss of information in the input representation.
3. **Sparsity:** it represents the fraction of weights set to zero during the discretization process. A fully connected layer can introduce a significant amount of redundancy and noise; removing less informative connections by increasing the sparsity leads to improved generalization.

4. **Positive ratio:** proportion of positive connections. The best results are expected when the number of positive and negative connections are balanced.

2 Dataset

In this project, the implementation at https://github.com/SandroErba/From-CNN-to-SNPS was tested in the context of DNA sequences. In particular, graphical representations of bacterial genomes, in the form of frequency chaos game representation, were given in input. The aim was to test whether an SN P system can be applied to the classification of genomic data.

The dataset comprised of bacterial complete RefSeq genomes (chromosome only) from four different genera: Aeromonas, Citrobacter, Streptococcus, Helicobacter; genomes were downloaded from the NCBI Genomes resource (https://www.ncbi.nlm.nih.gov/datasets/genome/). Table 1 summarizes the available data.

Genus	**N. genomes**	**Avg. length**	**Min. length**	**Max. length**
Aeromonas	428	4.8Mbp	3.9Mbp	5.6Mbp
Citrobacter	537	5.2Mbp	4.4Mbp	6.2Mbp
Helicobacter	514	1.6Mbp	1.4Mbp	2.6Mbp
Streptococcus	817	2.0Mbp	1.7Mbp	2.6Mbp

Table 1: Information about the downloaded genomes, including the genus name, the average, minimum and maximum genome length.

To have quite balanced classes, 500 random genomes from the set of Streptococcus genomes were selected. FCGRs were produced for each genome using the R package **kaos** and stored in the form of matrices, first with resolution 28 and then 100.

3 Experimental results

The Python code was modified in order to take in input the path to the folder where the DNA graphical representations are stored, which are expected to be found in different subfolders depending on the class they belong to. The configuration file was also modified to take into account the number of classes, the size of the training and test sets and the ternarization method (set at 1 to include the sparsity and positive ratio parameters). In this modified

implementation, the input dataset is automatically split in the two sets assigning $\frac{3}{4}$ to the training and the remaining $\frac{1}{4}$ to the test.

To classify images, the SN P System requires pixel intensities ranging from 0 to 255 (8-bit grayscale encoding). However, the FCGR values represent the number of points from the CGR in each area of the grid (the size depends on the resolution); these counts can easily exceed 255, which was confirmed by visual inspection of our data. To make the FCGR compatible with the requirements, we implemented a scaling process acting on the input dataset, which transforms the matrix counts into integers in the range [0,255]. Since the network learns from the high values (corresponding to lighter shades of grey, up to white), the higher the number of points in a cell the higher the pixel intensity (which is exactly the opposite we would see by plotting the original matrix, as shown in Figure 5 and Figure 6). In particular, each matrix is scaled separately, to avoid the effect of outliers. Each value of the matrix is divided by the maximum value of the matrix and multiplied by 255. Additionally, for the classification to work properly, classes were labeled using integers.

3.1 Results with images 28×28

The first set of results refer to FCGRs produced with resolution 28, which matched the size of the images in the MNIST dataset. Figure 5 shows the FCGR of two genomes in grayscale, belonging respectively to the genera of Helicobacter and Aeromonas; darker cells correspond to higher values.

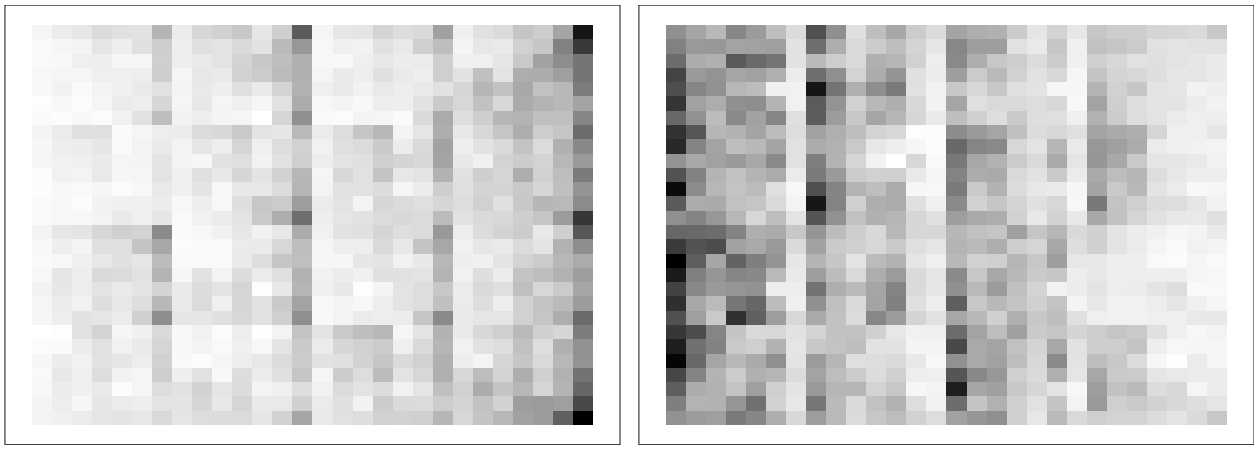


Figure 5: FCGR of two genomes, respectively belonging to the genus of Helicobacter (left) and Aeromonas (right); resolution set at 28.

Table 2 shows the results of the classification using predifined parameters ($q\text{-}range = 5$,

sparsity = 0.8, *positive ratio* = 0.1). All the configurations performed with very high

Configuration	**Accuracy**
SNPS svm	0.993407
SNPS svm with importance	0.993407
real weights svm	0.995604
SNPS logreg	0.991209
SNPS logreg with importance	0.991209
real weights logreg	0.997802
SNPS ensemble	0.993407
SNPS ensemble with importance	0.995604

Table 2: Accuracy of 8 configurations, which represent different ways of integrating the SNPS with the weight matrices produced by the support vector machine and logistic regression models.

accuracy, in all cases higher than 99%.

In the analysis by Erba et al. [4], the same resolution and parameters were used; the dataset in our analysis allowed us to achieve even higher accuracy. This is probably due to the higher amount of information associated with each image, which, compared to the MNIST, contain a lot of details and shades of gray, therefore giving more discriminative power to the model.

We modified the parameters of the configuration file to understand how the results would be affected. We varied the q-range and sparsity, modifying one value at a time; we wanted to see how the model would perform with lower amount of information, on one hand, and after introducing some noise, on the other hand. Table 3 shows the results after setting $q\text{-}range = \{2, 3, 4\}$, and Table 4 the results for $sparsity = \{0.2, 0.4, 0.6\}$

Configuration	**Acc.** $q\text{-}range = 2$	**Acc.** $q\text{-}range = 3$	**Acc.** $q\text{-}range = 4$
SNPS svm	0.865934	0.975824	0.967033
SNPS svm with importance	0.841758	0.914286	0.995604
real weights svm	0.745055	0.993407	0.995604
SNPS logreg	0.874725	0.982418	0.995604
SNPS logreg with importance	0.879121	0.989011	0.995604
real weights logreg	0.773626	0.993407	0.997802
SNPS ensemble	0.883516	0.986813	0.995604
SNPS ensemble with importance	0.83956	0.989011	0.995604

Table 3: Accuracy of the 8 configurations after modifying the q-range.

As for the MNIST dataset, low q-range deteriorates the accuracy of the model; however, the average accuracy is still around 80%, showing that the model can be effective even when the parameters are not optimal.

Configuration	**Acc.** $sparsity = 0.2$	**Acc.** $sparsity = 0.4$	**Acc.** $sparsity = 0.6$
SNPS svm	0.995604	0.995604	0.995604
SNPS svm with importance	0.997802	0.995604	0.995604
real weights svm	0.995604	0.995604	0.995604
SNPS logreg	0.991209	0.995604	0.995604
SNPS logreg with importance	0.995604	0.995604	0.995604
real weights logreg	0.997802	0.997802	0.997802
SNPS ensemble	0.995604	0.995604	0.995604
SNPS ensemble with importance	0.995604	0.995604	0.995604

Table 4: Accuracy of the 8 configurations after modifying the sparsity level.

Modifying the sparsity level, instead, has no effect on the performance of the model. This is probably due to the structure of the images; adding more connections to the model does not introduce noise, rather it only increases the level of detail.

3.2 Results with images 100×100

Figure 6 shows the same two genomes as in Figure 5, this time with resolution 100.

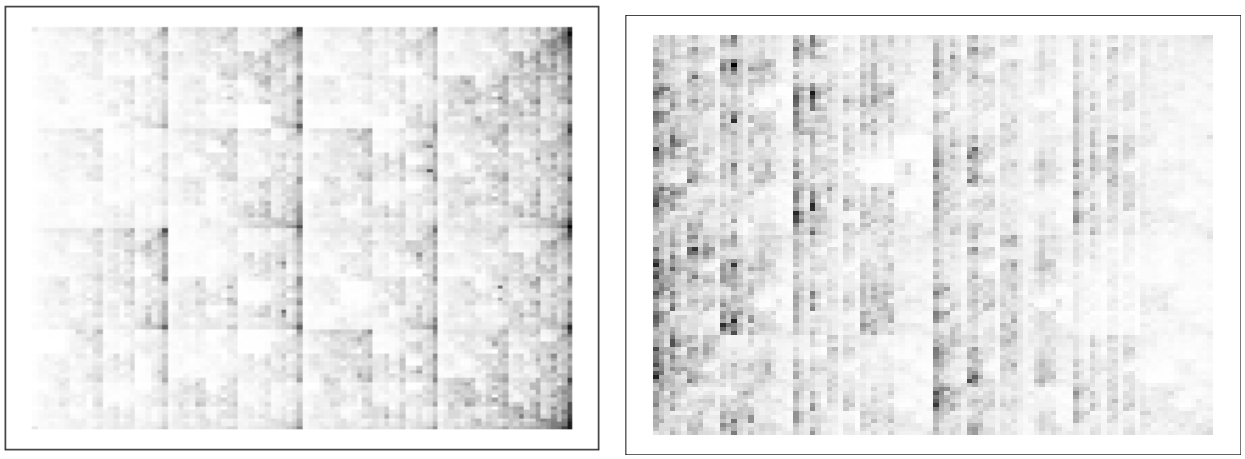


Figure 6: FCGR of two genomes, respectively belonging to the genus of Helicobacter (left) and Aeromonas (right); resolution set at 100.

We can observe that the images have higher complexity and patterns appear with more detail, even though they were already visible at resolution 28.

For the classification, the configuration file were kept with the original values, namely $q\text{-}range = 5$, $sparsity = 0.8$ and $positive\ ratio = 0.1$. Table 5 shows the results.

The model performs with accuracy above 99% for all configurations, obtaining results

Configuration	Accuracy
SNPS svm	0.99596
SNPS svm with importance	0.99596
real weights svm	0.99596
SNPS logreg	0.99596
SNPS logreg with importance	0.99596
real weights logreg	0.99596
SNPS ensemble	0.99596
SNPS ensemble with importance	0.99596

Table 5: Accuracy of the 8 configurations using images with resolution 100.

very similar to those obtained for images 28×28. This shows that the model performs accurately even when more complexity is added in input.

4 Conclusions and outlook

In this project we studied the structure of SN P systems, how to encode a DNA sequence in a chaos game representation, and how to classify such representations using an SN P system mimicking the main mechanisms of a CNN.

The dataset comprised genomes from four different genera, which were converted into FCGRs using the R package **kaos**. The implementation we used, modified to take into account the new type of input, obtained very good performances; on average, the accuracy was higher than 99%. Setting different parameters, such as the q-range, slightly affected the performance, which was significantly lower only when the q-range was equal to 2. Increasing the resolution of the images from 28 to 100 did not affect, in this case, the performance of the model, nor did decreasing the sparsity level.

Erba et al. [4] suggested that this implementation may not be suitable for complex datasets (compared to the MNIST). The feature extraction process uses simple edge-detection kernels, which cannot fully represent complex pattern. We may consider graphical representations of genomes to be associated to complex patterns with high degree of detail; the model, however, was still able to perform with high accuracy, discriminating genomes almost perfectly. By looking at Figure 5 and Figure 6, genomes from different genera do appear differently; this is be due to different k-mer frequencies, but also to the differences in size, as shown in Table 1.

As a future perspective, we would suggest to test the implementation on a larger number

of taxa, and understand how the model behaves when genome sizes are more uniform, which would probably require the model to extract more subtle patterns from the images. Additionally, the discriminative power of the model could be tested at lower resolutions (e.g. at resolution 8), to see if it can accurately classify genomes when less information is provided. Given that studies such as the one by Hammad et al. [6], who implemented a deep learning method for the classification of FCGRs to distinguish COVID-19 from other HCoV, used deep feature extraction layers, it would be interesting to see how this simpler model performs in the classification of viral genomes as well.

References

[1] G. Păun, M. J. Pérez-Jiménez, and A. Salomaa. Spiking Neural P systems: An early survey. *International Journal of Foundations of Computer Science*, 18(03):435–455, 2007. doi: 10.1142/S0129054107004772.

[2] H. J. Jeffrey. Chaos game representation of gene structure. *Nucleic acids research*, 18(8): 2163–2170, 1990.

[3] H. F. Löchel and D. Heider. Chaos game representation and its applications in bioinformatics. *Computational and Structural Biotechnology Journal*, 19:6263–6271, 2021. doi: 10.1016/j.csbj.2021.11.008.

[4] S. Erba, G. Franco, and C. Zandron. Towards a lightweight Spiking Neural P system for image recognition. In *Proceedings of the 23rd International Conference on Unconventional Computation and Natural Computation*. Springer Lecture Notes in Computer Science Series (LNCS). To appear.

[5] L. Deng. The MNIST database of handwritten digit images for machine learning research. *IEEE Signal Processing Magazine*, 29(6):141–142, 2012.

[6] M. S. Hammad, V. F. Ghoneim, M. S. Mabrouk, and W. I. Al-atabany. A hybrid deep learning approach for COVID-19 detection based on genomic image processing techniques. *Scientific Reports*, 13, 2023. doi: 10.1038/s41598-023-30941-0.